

C++ Standard Library "Review" Issues Resolved in Kona

Doc. no. P0610R0

Date: Revised 2017-03-04 at 22:03:08 UTC

Project: Programming Language C++

Reply to: Marshall Clow <lwgchair@gmail.com>

Review Issues

[2676](#). Provide `filesystem::path` overloads for *File-based streams*

Section: 27.9 [file.streams] **Status:** [Review](#) **Submitter:** Beman Dawes **Opened:** 2016-03-16 **Last modified:** 2016-12-23

Priority: 2

View all other [issues](#) in [file.streams].

View all issues with [Review](#) status.

Discussion:

The constructor and open functions for *File-based streams* in 27.9 [file.streams] currently provide overloads for `const char*` and `const string&` arguments that specify the path for the file to be opened. With the addition of the File System TS to the standard library, these constructors and open functions need to be overloaded for `const filesystem::path&` so that file-based streams can take advantage of class `filesystem::path` features such as support for strings of character types `wchar_t`, `char16_t`, and `char32_t`.

The `const filesystem::path& p` overload for these functions is like the existing `const string&` overload; it simply calls the overload of the same function that takes a C-style string.

For operating systems like POSIX that traffic in `char` strings for filenames, nothing more is required. For operating systems like Windows that traffic in `wchar_t` strings for filenames, an additional C-style string overload is required. The overload's character type needs to be specified as `std::filesystem::path::value_type` to also support possible future operating systems that traffic in `char16_t` or `char32_t` characters.

Not recommended:

Given the proposed constructor and open signatures taking `const filesystem::path&`, it would in theory be possible to remove some of the other signatures. This is not proposed because it would break ABI's, break user code depending on user-defined automatic conversions to the existing argument types, and invalidate existing documentation, books, and tutorials.

Implementation experience:

The Boost Filesystem library has provided header `<boost/filesystem/fstream.hpp>` implementing the proposed resolution for over a decade.

The Microsoft/Dinkumware implementation of standard library header `<fstream>` has provided the `const wchar_t*` overloads for many years.

[2016-08-03 Chicago]

Fri PM: Move to Review

Proposed resolution:

At the end of 27.9.1 File streams [fstreams] add a paragraph:

In this subclause, member functions taking arguments of `const std::filesystem::path::value_type*` shall only be provided on systems where `std::filesystem::path::value_type` ([class.path]) is not `char`. [Note: These functions enable class path support for systems with a wide native path character type, such as `wchar_t`. — end note]

Change 27.9.1.1 Class template basic_filebuf [filebuf] as indicated:

```
basic_filebuf<charT,traits>* open(const char* s,
```

```

        ios_base::openmode mode);
basic_filebuf<charT,traits>* open(const std::filesystem::path::value_type* s,
        ios_base::openmode mode); // wide systems only; see [fstreams]
basic_filebuf<charT,traits>* open(const string& s,
        ios_base::openmode mode);
basic_filebuf<charT,traits>* open(const filesystem::path& p,
        ios_base::openmode mode);

```

Change 27.9.1.4 Member functions [filebuf.members] as indicated:

```

basic_filebuf<charT,traits>* open(const char* s,
        ios_base::openmode mode);
basic_filebuf<charT,traits>* open(const std::filesystem::path::value_type* s,
        ios_base::openmode mode); // wide systems only; see [fstreams]

```

To 27.9.1.4 Member functions [filebuf.members] add:

```

basic_filebuf<charT,traits>* open(const filesystem::path& p,
        ios_base::openmode mode);

```

Returns: open(p.c_str(), mode);

Change 27.9.1.6 Class template basic_ifstream [ifstream] as indicated:

```

explicit basic_ifstream(const char* s,
        ios_base::openmode mode = ios_base::in);
explicit basic_ifstream(const std::filesystem::path::value_type* s,
        ios_base::openmode mode = ios_base::in); // wide systems only; see [fstreams]
explicit basic_ifstream(const string& s,
        ios_base::openmode mode = ios_base::in);
explicit basic_ifstream(const filesystem::path& p,
        ios_base::openmode mode = ios_base::in);
...
void open(const char* s,
        ios_base::openmode mode = ios_base::in);
void open(const std::filesystem::path::value_type* s,
        ios_base::openmode mode = ios_base::in); // wide systems only; see [fstreams]
void open(const string& s,
        ios_base::openmode mode = ios_base::in);
void open(const filesystem::path& p,
        ios_base::openmode mode = ios_base::in);

```

Change 27.9.1.7 basic_ifstream constructors [ifstream.cons] as indicated:

```

explicit basic_ifstream(const char* s,
        ios_base::openmode mode = ios_base::in);
explicit basic_ifstream(const std::filesystem::path::value_type* s,
        ios_base::openmode mode = ios_base::in); // wide systems only; see [fstreams]

```

To 27.9.1.7 basic_ifstream constructors [ifstream.cons] add:

```

explicit basic_ifstream(const filesystem::path& p,
        ios_base::openmode mode = ios_base::in);

```

Effects: the same as basic_ifstream(p.c_str(), mode).

Change 27.9.1.9 Member functions [ifstream.members] as indicated:

```

void open(const char* s,
        ios_base::openmode mode = ios_base::in);
void open(const std::filesystem::path::value_type* s,
        ios_base::openmode mode = ios_base::in); // wide systems only; see [fstreams]

```

To 27.9.1.9 Member functions [ifstream.members] add:

```

void open(const filesystem::path& p,
        ios_base::openmode mode = ios_base::in);

```

Effects: calls open(p.c_str(), mode).

Change 27.9.1.10 Class template basic_ofstream [ofstream] as indicated:

```

explicit basic_ofstream(const char* s,
        ios_base::openmode mode = ios_base::out);
explicit basic_ofstream(const std::filesystem::path::value_type* s,
        ios_base::openmode mode = ios_base::out); // wide systems only; see [fstreams]
explicit basic_ofstream(const string& s,
        ios_base::openmode mode = ios_base::out);

```

```
explicit basic_ofstream(const filesystem::path& p,
    ios_base::openmode mode = ios_base::out);
...
void open(const char* s,
    ios_base::openmode mode = ios_base::out);
void open(const std::filesystem::path::value_type* s,
    ios_base::openmode mode = ios_base::out); // wide systems only; see [fstreams]
void open(const string& s,
    ios_base::openmode mode = ios_base::out);
void open(const filesystem::path& p,
    ios_base::openmode mode = ios_base::out);
```

Change 27.9.1.11 basic_ofstream constructors [ofstream.cons] as indicated:

```
explicit basic_ofstream(const char* s,
    ios_base::openmode mode = ios_base::out);
explicit basic_ofstream(const std::filesystem::path::value_type* s,
    ios_base::openmode mode = ios_base::out); // wide systems only; see [fstreams]
```

To 27.9.1.11 basic_ofstream constructors [ofstream.cons] add:

```
explicit basic_ofstream(const filesystem::path& p,
    ios_base::openmode mode = ios_base::out);

Effects: the same as basic_ofstream(p.c_str(), mode).
```

Change 27.9.1.13 Member functions [ofstream.members] as indicated:

```
void open(const char* s,
    ios_base::openmode mode = ios_base::out);
void open(const std::filesystem::path::value_type* s,
    ios_base::openmode mode = ios_base::out); // wide systems only; see [fstreams]
```

To 27.9.1.13 Member functions [ofstream.members] add:

```
void open(const filesystem::path& p,
    ios_base::openmode mode = ios_base::out);

Effects: calls open(p.c_str(), mode).
```

Change 27.9.1.14 Class template basic_fstream [fstream] as indicated:

```
explicit basic_fstream(const char* s,
    ios_base::openmode mode = ios_base::in|ios_base::out);
explicit basic_fstream(const std::filesystem::path::value_type* s,
    ios_base::openmode mode = ios_base::in|ios_base::out); // wide systems only; see [fstreams]
explicit basic_fstream(const string& s,
    ios_base::openmode mode = ios_base::in|ios_base::out);
explicit basic_fstream(const filesystem::path& p,
    ios_base::openmode mode = ios_base::in|ios_base::out);
...
void open(const char* s,
    ios_base::openmode mode = ios_base::in|ios_base::out);
void open(const std::filesystem::path::value_type* s,
    ios_base::openmode mode = ios_base::in|ios_base::out); // wide systems only; see [fstreams]
void open(const string& s,
    ios_base::openmode mode = ios_base::in|ios_base::out);
void open(const filesystem::path& p,
    ios_base::openmode mode = ios_base::in|ios_base::out);
```

Change 27.9.1.15 basic_fstream constructors [fstream.cons] as indicated:

```
explicit basic_fstream(const char* s,
    ios_base::openmode mode = ios_base::in|ios_base::out);
explicit basic_fstream(const std::filesystem::path::value_type* s,
    ios_base::openmode mode = ios_base::in|ios_base::out); // wide systems only; see [fstreams]
```

To 27.9.1.15 basic_fstream constructors [fstream.cons] add:

```
explicit basic_fstream(const filesystem::path& p,
    ios_base::openmode mode = ios_base::in|ios_base::out);

Effects: the same as basic_fstream(p.c_str(), mode).
```

Change 27.9.1.17 Member functions [fstream.members] as indicated:

```
void open(const char* s,
    ios_base::openmode mode = ios_base::in|ios_base::out);
```

```
void open(const std::filesystem::path::value_type* s,  
ios_base::openmode mode = ios_base::in|ios_base::out); // wide systems only; see [fstreams]
```

To 27.9.1.17 Member functions [fstream.members] add:

```
void open(const filesystem::path& p,  
ios_base::openmode mode = ios_base::in|ios_base::out);
```

Effects: calls `open(p.c_str(), mode)`.

2790. Missing specification of `istreambuf_iterator::operator->`

Section: 24.6.3 [istreambuf.iterator] **Status:** [Review](#) **Submitter:** Jonathan Wakely **Opened:** 2016-11-09 **Last modified:** 2017-02-02

Priority: 3

View other [active issues](#) in [istreambuf.iterator].

View all other [issues](#) in [istreambuf.iterator].

View all issues with [Review](#) status.

Discussion:

Addresses GB 59

There is no specification for `istreambuf_iterator::operator->`. This operator appears to have been added for C++11 by LWG issue [659](#), which gave the signature, but also lacked specification.

[2016-11-08, Jonathan comments and suggests wording]

There is no good option here, and implementations either return `nullptr`, or return the address of a temporary, or don't even provide the member at all. We took polls to decide whether to remove `istreambuf_iterator::operator->`, or specify it to return `nullptr`, and the preferred option was to remove it. It was noted that in the Ranges TS input iterators no longer require `operator->` anyway, and the library never tries to use it.

[Issues Telecon 16-Dec-2016]

Move to Review

Proposed resolution:

This wording is relative to N4606.

This reverts LWG [659](#).

1. Remove the note in paragraph 1 of 24.6.3 [istreambuf.iterator]:

The class template `istreambuf_iterator` defines an input iterator (24.2.3) that reads successive characters from the streambuf for which it was constructed. `operator*` provides access to the current input character, if any. ~~[Note: `operator->` may return a proxy. — end note]~~ Each time `operator++` is evaluated, the iterator advances to the next input character. [...]

2. Remove the member from the class synopsis in 24.6.3 [istreambuf.iterator]:

```
charT operator*() const;  
pointer operator->() const;  
istreambuf_iterator& operator++();  
proxy operator++(int);
```

2796. tuple should be a literal type

Section: 20.5.1 [tuple.general] **Status:** [Review](#) **Submitter:** Jonathan Wakely **Opened:** 2016-11-09 **Last modified:** 2017-02-02

Priority: 2

View other [active issues](#) in [tuple.general].

View all other [issues](#) in [tuple.general].

View all issues with [Review](#) status.

Discussion:

Addresses US 109

`tuple` should be a literal type if its elements are literal types; it fails because the destructor is not necessarily trivial. It should follow the form of optional and variant, and mandate a trivial destructor if all types in `Types...` have a trivial destructor. It is not clear if `pair` has the same issue, as `pair` specifies data members `first` and `second`, and appears to have an implicitly declared and defined destructor.

Suggested resolution:

Document the destructor for `tuple`, and mandate that it is trivial if each of the elements in the `tuple` has a trivial destructor. Consider whether the same specification is needed for `pair`.

[2016-11-09, Jonathan provides wording]

[Issues Telecon 16-Dec-2016]

Move to Review; we think this is right, but are awaiting implementation experience.

Proposed resolution:

This wording is relative to N4606.

1. Add a new paragraph after 20.4.2 [pairs.pair] p2:

-2- The defaulted move and copy constructor, respectively, of `pair` shall be a `constexpr` function if and only if all required element-wise initializations for copy and move, respectively, would satisfy the requirements for a `constexpr` function. The destructor of `pair` shall be a trivial destructor if `(is_trivially_destructible v<T1> && is_trivially_destructible v<T2>)` is true.

2. Add a new paragraph after the class synopsis in 20.5.3 [tuple.tuple]:

-?- The destructor of `tuple` shall be a trivial destructor if `(is_trivially_destructible v<Types> && ...)` is true.